Course Code: **IDC 222**

Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

Procedural Oriented Programming

Procedural Oriented Programming

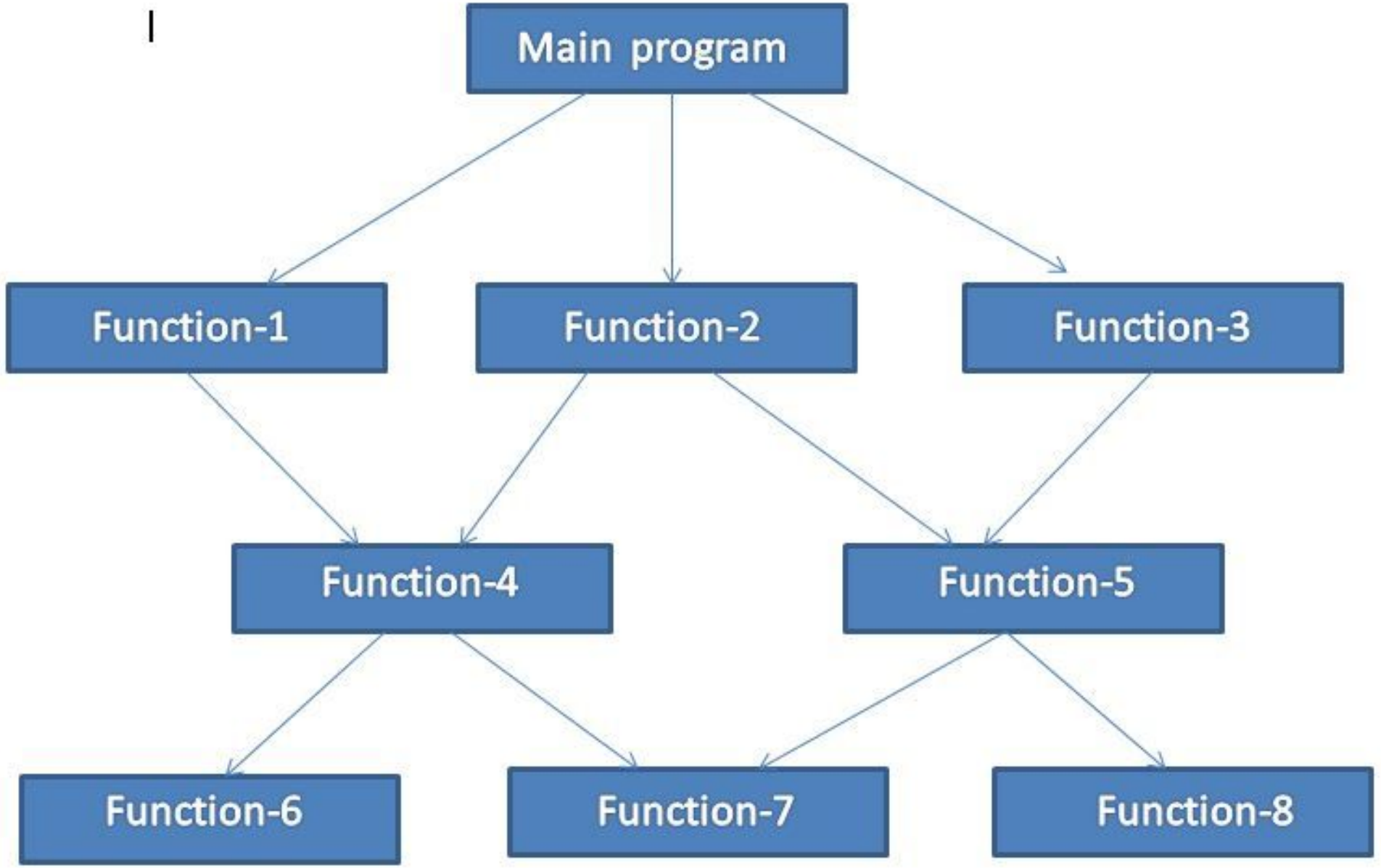
Procedural Programming is a programming paradigm derived from **structured programming**, which is based on the concept of **procedures or functions.**

- A **procedure (function**) is a block of code that performs a specific task.
- A program is divided into **small functions.**
- These functions are executed **step by step** to solve a problem.
- A function can be called:
    - by **another function**
    - or **by itself** (**recursion**)

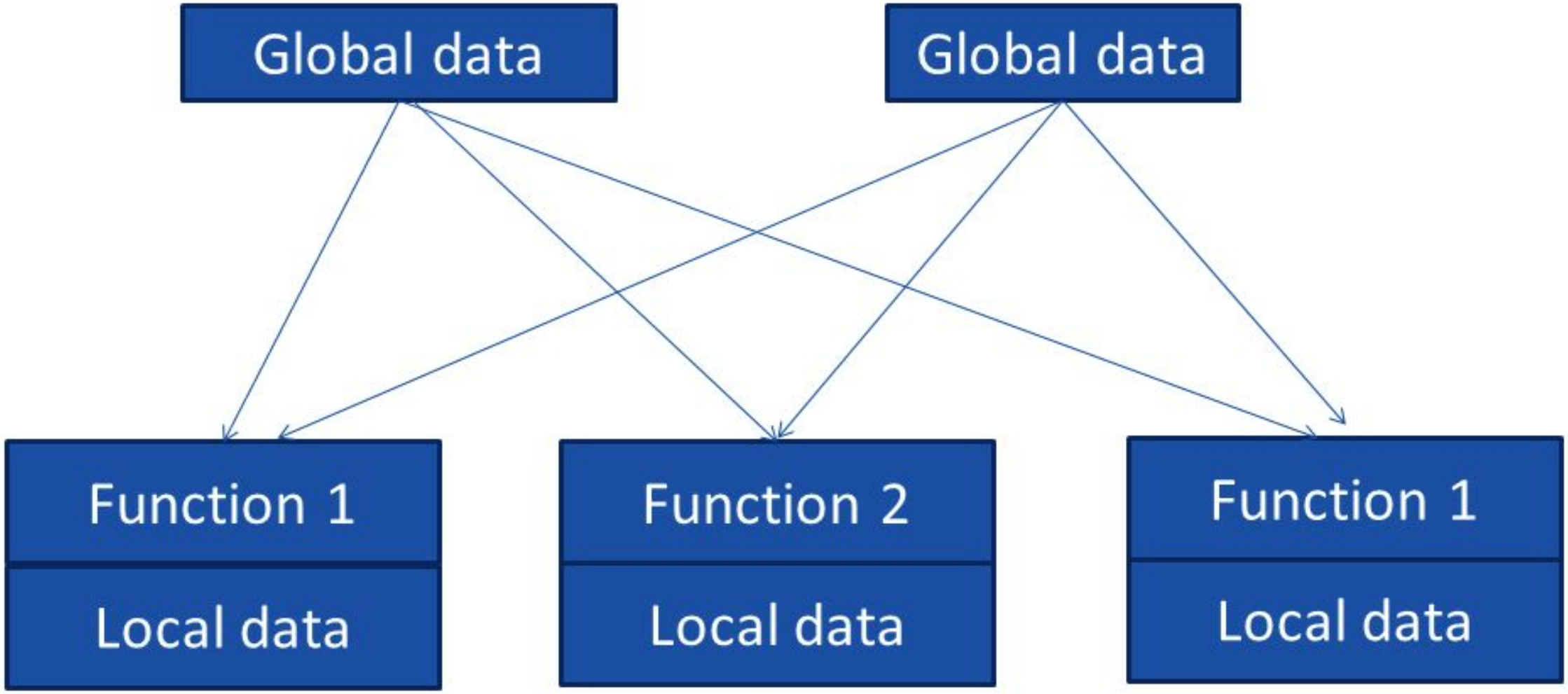
Procedural Programming Languages are:-

BASIC, FORTRAN, COBOL, ALGOL, Pascal, and C.

Structure of procedural oriented programming



Relationship of data and function in procedural programming



Characteristics of Procedural Programming

- Emphasis is on **doing things** (focus on **algorithms**).
- Large programs are divided into **smaller units called functions.**
- Most functions **share global data.**
- Data moves **freely from function to function.**
- Functions **transform data** from one form to another.
- Follows a **Top-Down approach** in program design.

Object Oriented Programming

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that is based on the concept of **objects.**

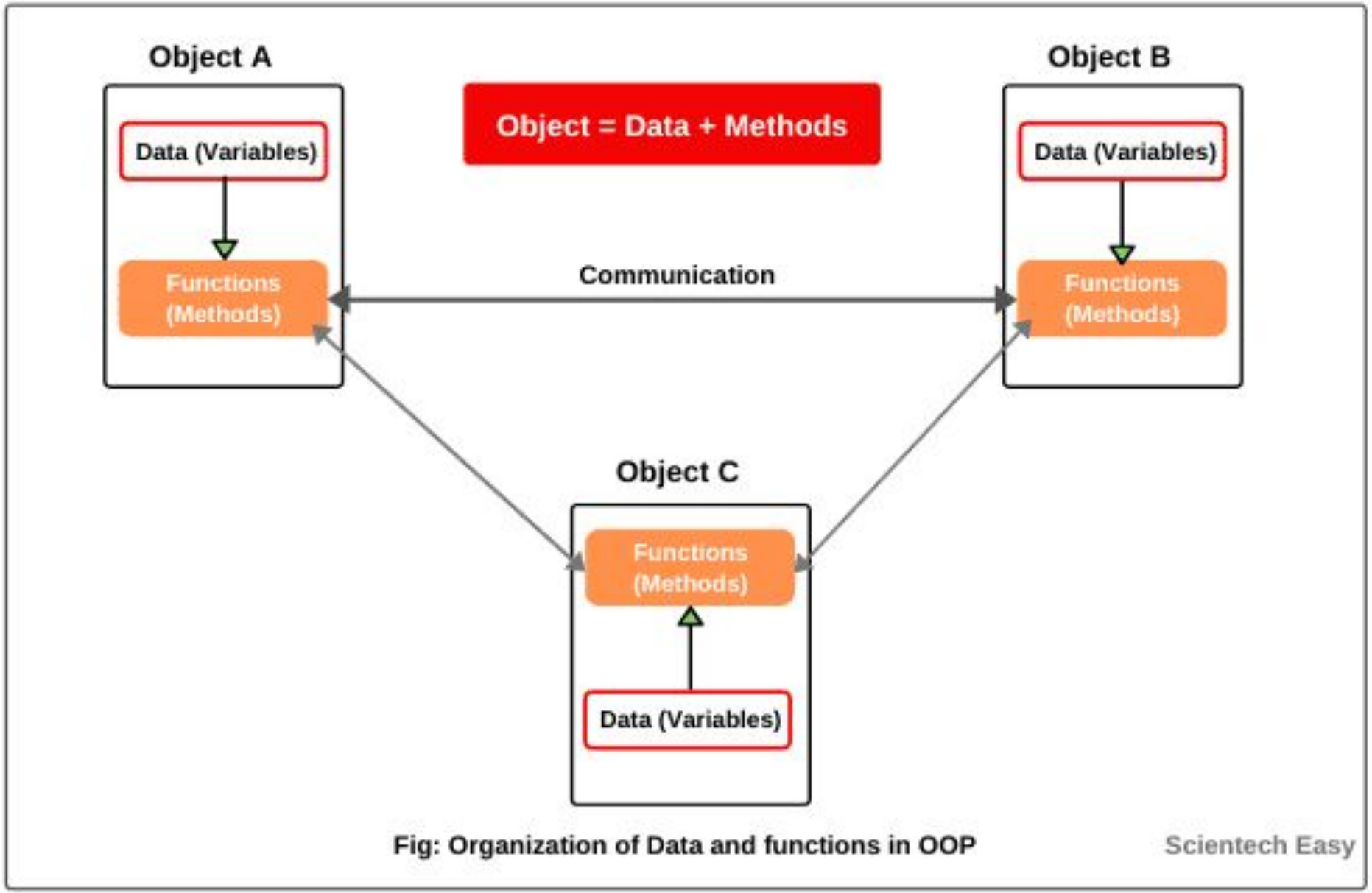
- OOP is a **methodology** used to design programs using **classes and objects**.
- An object contains:
  - **Data** in the form of attributes **(variables)**
  - **Code** in the form of methods **(functions)**
- Programs are designed by modeling **real-world entities** as objects.
- Objects **interact with each other** to perform tasks.
- Objects are **instances of classes**, and the class defines the **type and behavior** of the object

Object Oriented Programming Languages are:-

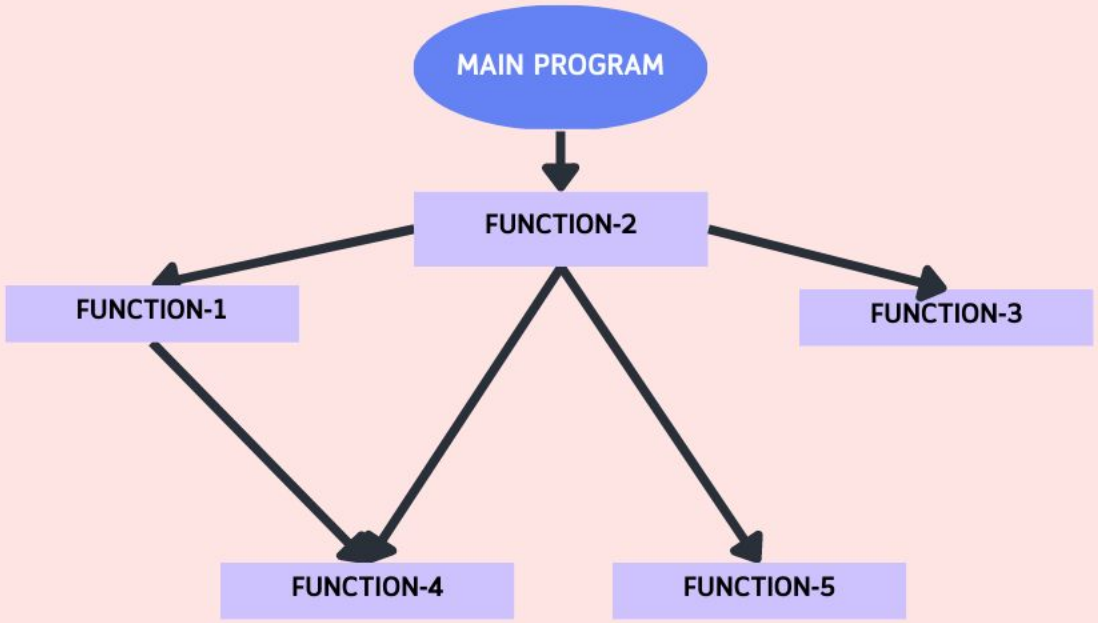
Java, C++, C#, Python, PHP, JavaScript,

Ruby, Perl, Objective-C, Dart, Swift, Scala.

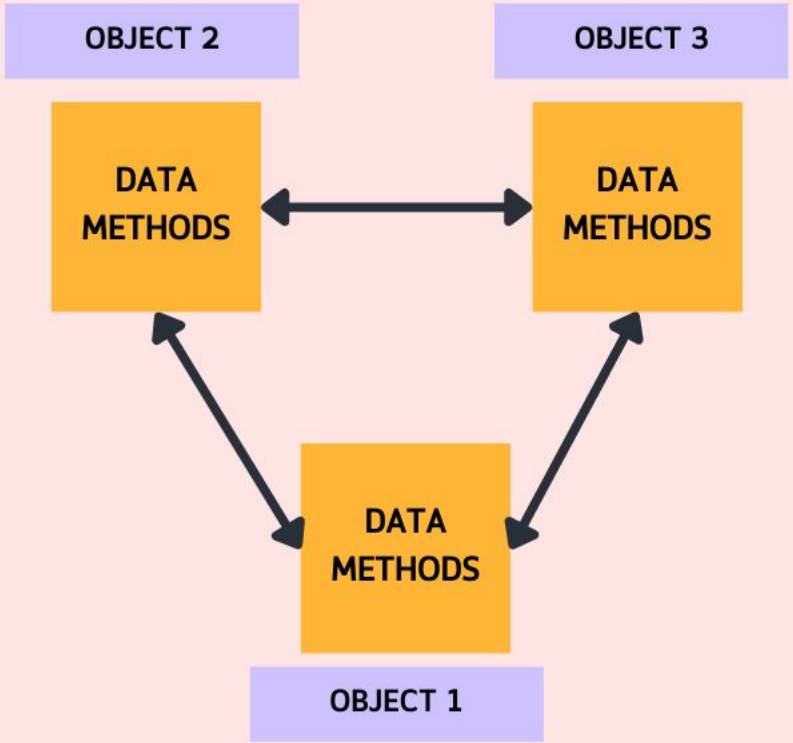
Organization of data and functions in OOPS



Procedural Programming



Object Orient Programming



POP vs OOP

Procedural Oriented Programming

- program is divided into small parts called ***functions***.
- It follows a ***top-down approach***.
- It does not have any proper way of hiding data so it is ***less secure***.
- In POP, the function is more important than the data.
- POP is based on the ***unreal world***.
- It is not very suitable for solving any big or complex problems.

Object-Oriented Programming

- program is divided into small parts called ***objects***.
- It follows a ***bottom-up approach***.
- It provides data hiding so it is ***more secure***.
- In OOP, data is more important than function.
- OOP is based on the ***real world***.
- It is suitable for solving any big or complex problems.

Basic OOP Concepts

Basic OOP Concepts in C++

1. Classes ✅
2. Objects ✅
3. Constructors ✅
4. Inheritance
5. Polymorphism
6. Abstraction
7. Encapsulation

we will focus only on:

- Classes
- Objects
- Constructors

Understanding these concepts is sufficient to learn and use STL effectively.

Class

- A class is a **blueprint or template** used to create objects.
- It is a **user-defined data type**.
  - **Data members** → variables of the class
  - **Member functions** → functions used to manipulate data members
- **Data members** define the properties of the class.
- **Member functions** define the behavior of the class.
- The members of a class are accessed by creating an **object (instance)** of the class.
- A class **does not consume memory**; memory is allocated only **when an object is created.**

Syntax of a Class

keyword User-defined name

```
class classname
{
    access specifier:    // Can be private, public, protected

    Data members;        // Variables to be used
    Member functions;    // Methods to access data members
};
```

Note: A class definition always ends with a semicolon (;).

Defining a Class

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
   private:
      int rollno;
      string name;
      float marks;

   public:
      void input();
      void display();

};
```

// Data members

// Member functions

Example: BankAccount Class

```
class BankAccount
{
   private:
        int accountNumber;
        string holderName;
        float balance;

   public:
        void setAccount(int accNo, string name, float bal);
        void deposit(float amount);
        void withdraw(float amount);
        void checkBalance();
};
```

// Data members

// Member functions

Explanation:

Class Name: **BankAccount** .

Access Specifier:

- **private** → protects sensitive data (account number, holder name, balance)
- **public** → allows controlled access to data using member functions

Data Members:

- **accountNumber** → stores account number
- **holderName** → stores account holder's name
- **balance** → stores current balance

Member Functions:

- **setAccount()** → initializes account details
- **deposit()** → adds money to the account
- **withdraw()** → removes money from the account
- **checkBalance()** → displays the current balance

Contd…

- A class can have **multiple objects.**
- All objects of a class **share common properties and behavior.**
- Each object represents a **real-world entity** created from the class.

Example: **Book**

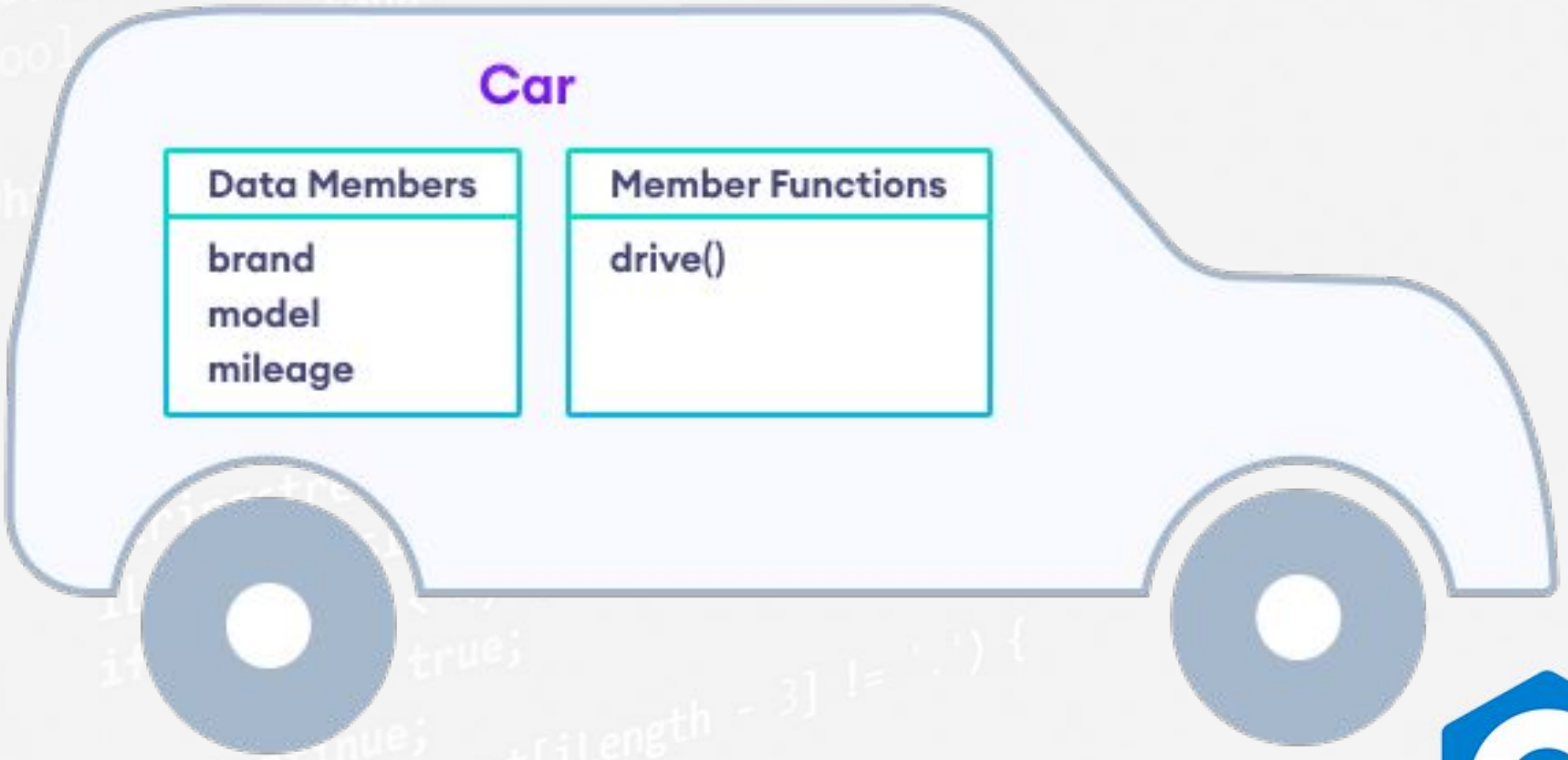
- A **class** acts as a blueprint for a **book**.
- It defines:

  Properties → title, author

  Behavior → reading
- An **object** is a **specific book** created using this blueprint.

  Example: "C++ Programming" book is an object of the Book class.

Example


Car
Data Members
brand
model
mileage
Member Functions
drive()

Properties

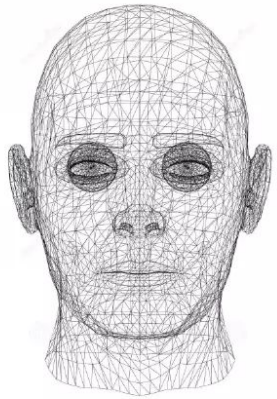
name
age
gender
city

Methods

walk ()
sleep ()
speak ()
eat ()

This is class Human

It is just a bluePrint



Properties

name : Ron
age : 15
gender : Male
city : Delhi

Methods

can walk ()
can sleep ()
can speak ()
can eat ()

This is an instance of the Human class

It exists in real and is implementation of class Human

Object

- An **object** is a **real-world entity** that has **state** and **behavior**.
- An object is an **instance of a class.**
- Multiple objects can be created from the **same class.**
- A class only defines the **specification** of an object.
- **Memory is allocated only when an object is created**, not when a class is defined.
- Objects **occupy memory** and have **unique addresses.**
- Objects are used to **access data members and member functions** of a class.

Class vs Object

Class → Blueprint or template

Object → Real instance created from the class

Object Declaration and Accessing Members

Object Declaration

Syntax:

```
ClassName objectName;
```

Example:

```
BankAccount acc1;
```

Accessing Class Members

- Members of a class are accessed using the **dot (.) operator.**

Syntax:

```
objectName.memberName;
```

Example:

```
acc1.checkBalance();
```

Here, `acc1` is an **object** and `checkBalance()` is a **member function** of **BankAccount** Class.

Example: Bank Account Object

Class: BankAccount

- Think of a class as a blueprint for a bank account.

Object

- A **BankAccount object** represents **a real bank account** of a **customer**.

State (Data Members)

These represent the **current state** of the object:

- **accountNumber**
- **holderName**
- **balance**

Each account object has **its own values** for these.

Behavior (Member Functions)

These represent **actions** the object can perform:

- **deposit()**
- **withdraw()**
- **checkBalance()**

Contd...

Memory Concept

- When the **class** `BankAccount` is defined → **no memory is allocated**
- When an **object** is created:

```
BankAccount acc1;
```

- Memory is allocated for `accountNumber`, `holderName`, and `balance`.

Multiple Objects from Same Class

```
BankAccount acc1;
BankAccount acc2;
```

- `acc1` and `acc2` are **different objects**
- Both follow the **same structure**
- But each has **different data values**
- Each object has a **unique memory address**

Contd...

Object Interaction Example

- **Customer** object wants to know bank balance
- It sends a message to the **Account** object:

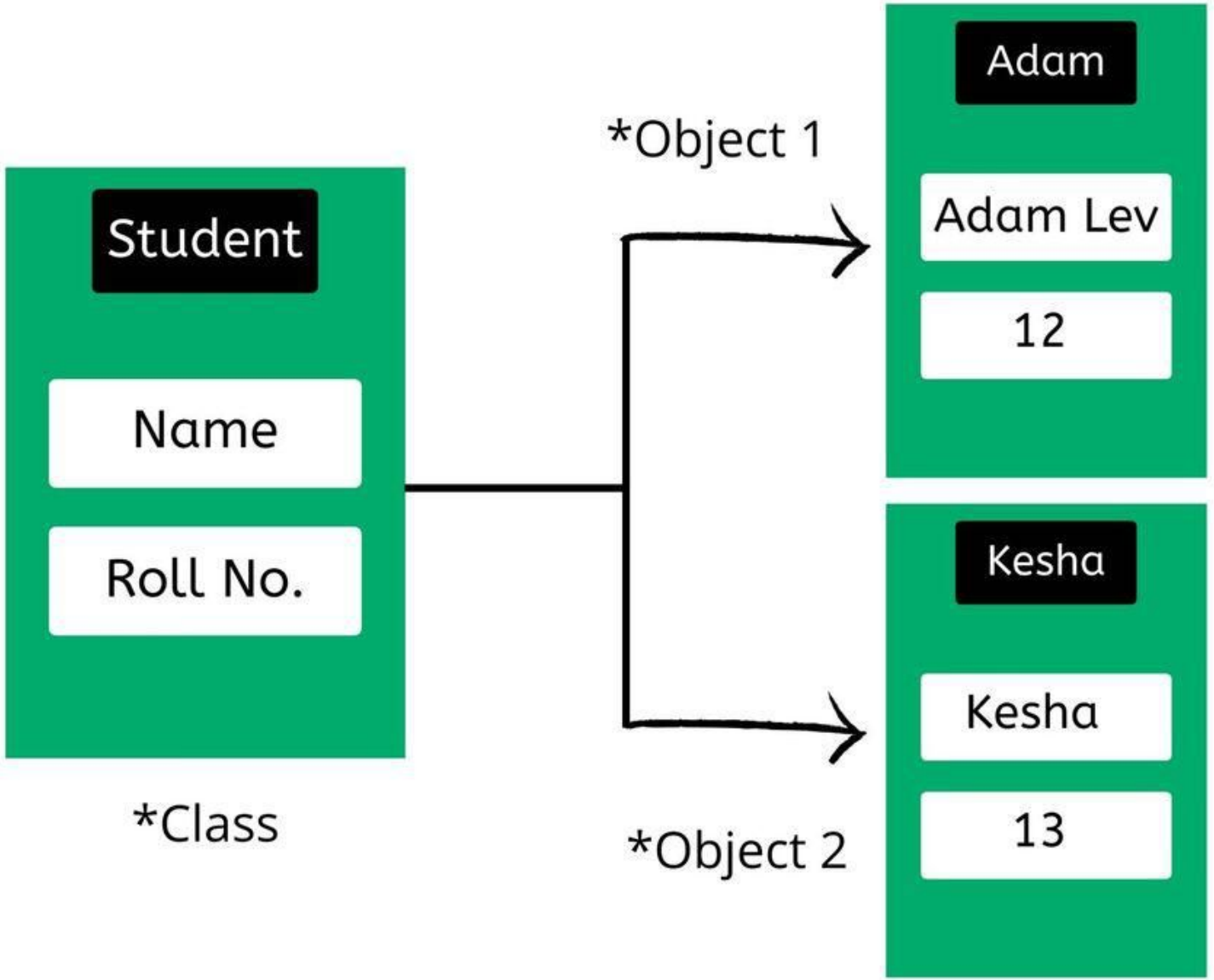
```
acc1.checkBalance();
```

- The account object responds with the balance.
- The customer does not know internal details of the account.

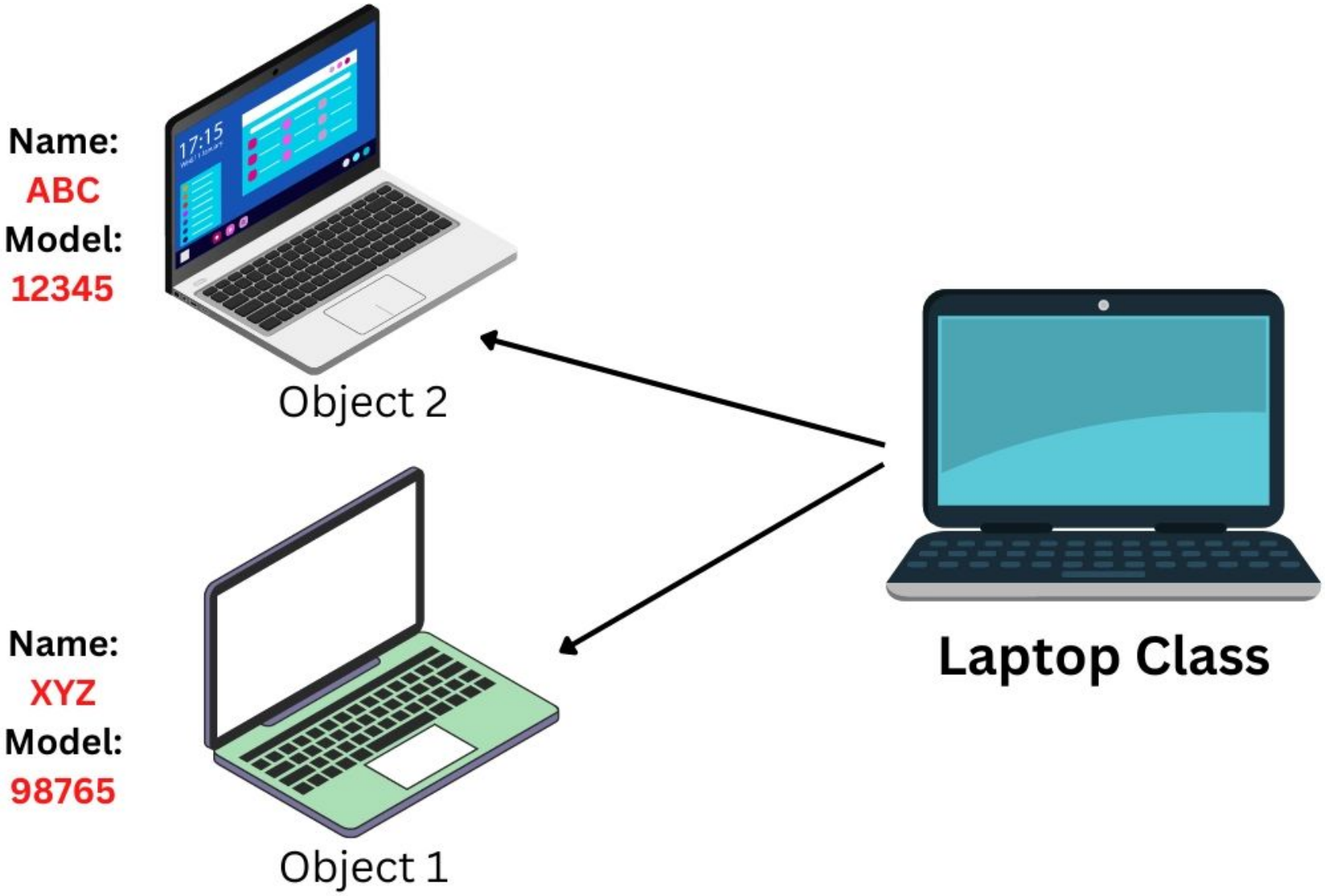
Why This is Object-Oriented?

- Data and functions are **bundled together**
- Objects interact using **well-defined interfaces**
- Internal data remains **hidden and protected**

Example of Objects



Example of Objects



Class Definition: Student

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student
{
  private:
    int rollno;
    string name;
    float marks;

  public:
    void getData()
    {
        cout << "Enter roll number: ";
        cin >> rollno;

        cout << "Enter name: ";
        cin >> ws;
        getline(cin, name);

        cout << "Enter marks: ";
        cin >> marks;
    }
    void display()
    {
        cout << "Roll Number: " << rollno << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};
```

Accessing Class Members Using Objects

```cpp
int main()
{
    Student s1;

    s1.getData();
    s1.display();

    return 0;
}
```

It removes all leading whitespace characters from the input buffer.

Whitespace includes:

- Space ' '
- Newline '\n'
- Tab '\t'

Class Definition: Student

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
   private:
        int rollno;
        string name;
        float marks;

   public:
        void setData(int r, string n, float m)
        {
            rollno = r;
            name = n;
            marks = m;
        }

        void display()
        {
            cout << "Roll Number: " << rollno << endl;
            cout << "Name: " << name << endl;
            cout << "Marks: " << marks << endl;
        }
};
```

Accessing Class Members Using Objects

```
int main()
{
    Student s1;

    s1.setData(101, "Rahul", 85.5);
    s1.display();
    return 0;
}
```

What will be the output?

```cpp
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    if (x > 5)
    {
        if (x > 8)
            cout << "A";
        else
            cout << "B";
    }
    return 0;
}
```

What will be the output?

```
#include <iostream>
using namespace std;

int main()
{
    int x = 2;

    switch (x)
    {
        case 1:
            cout << "One ";
        case 2:
            cout << "Two ";
        case 3:
            cout << "Three ";
        default:
            cout << "Default ";
    }

    return 0;
}
```

Access Specifiers in C++

1. Private

- Use **private** for **data members** (rollno, name, marks)
- Prevents direct access from outside the class
- Ensures **data hiding / encapsulation**

```
private:
    int rollno;
    string name;
    float marks;
```

2. Public

- Use **public** for member functions (getData(), displayData())
- Allows objects to access and modify private data safely
- Provides a controlled interface to interact with the class

```
public:
    void getData();
    void displayData();
```

Accessing Private and Public Members

```cpp
#include <iostream>
using namespace std;

class Person
{
  private:
     int age;          // private data member

  public:
     int id;           // public data member
};

int main()
{
   Person p;

   p.id = 101;

   cout << "ID: " << p.id << endl;

   p.age = 20;

   return 0;
}
```

Private members cannot be accessed directly

Public functions act as a bridge to access private data

Accessing Private and Public Members

```cpp
#include <iostream>
using namespace std;

class Person
{
  private:
     int age;          // private data member

  public:
     int id;           // public data member
};

int main()
{
   Person p;

   p.id = 101;

   cout << "ID: " << p.id << endl;

   p.age = 20;

   return 0;
}
```

Private members cannot be accessed directly

Public functions act as a bridge to access private data

✅ Allowed (public)

❌ Not allowed (private)

Accessing Private Members Using Public Functions (Without Parameter)

```cpp
#include <iostream>
using namespace std;

class Person
{
 private:
  int age;      // private data member

 public:
  void inputAge()   // public member function
  {
      cin >> age;
  }

  void showAge()
  {
      cout << "Age: " << age << endl;
  }
};

int main()
{
  Person p;

  // p.age = 20;    // ❌ Not allowed (private)

  p.inputAge();     // ✅ Allowed using public function
  p.showAge();

  return 0;
}
```

Important Note:

Private data members are accessed through public member functions.

Accessing Private Members Using Public Functions (With Parameter)

```cpp
#include <iostream>
using namespace std;

class Person
{
  private:
   int age;      // private data member

  public:
   void setAge(int a)    // public member function
   {
       age = a;
   }

   void showAge()
   {
       cout << "Age: " << age << endl;
   }
};

int main()
{
   Person p;

   // p.age = 20;    // ❌ Not allowed (private)

   p.setAge(20);     // ✅ Allowed using public function
   p.showAge();

   return 0;
}
```

Important Note:

Private data members are accessed through public member functions.

Member Functions Defined Outside the Class

- Member functions can be **declared inside the class** and **defined outside**
- **`::` (scope resolution operator)** tells the compiler which class the function belongs to
- This improves program **readability** and **code organization**, especially for large programs.

Syntax:

```
class ClassName
{
public:
   returnType functionName(parameters);
};

returnType ClassName ::functionName(parameters)
{
   // function definition
}
```

Example:

```
class Student
{
public:
   void display();
};

void Student :: display()
{
   cout << "Hello Student";
}
```

BankAccount Example : Member Functions Defined Outside the Class

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
private:
    int accountNumber;
    string holderName;
    float balance;

public:
    void setAccount(int accNo, string name, float bal);
    void deposit(float amount);
    void withdraw(float amount);
    void checkBalance();
};

void BankAccount :: setAccount(int accNo, string name, float bal)
{
    accountNumber = accNo;
    holderName = name;
    balance = bal;
}

void BankAccount :: deposit(float amount)
{
    balance = balance + amount;
    cout << "Amount deposited successfully." << endl;
}
```

```cpp
void BankAccount :: withdraw(float amount)
{
    balance = balance - amount;
    cout << "Amount withdrawn successfully." << endl;
}

void BankAccount :: checkBalance()
{
    cout << "Account Number: " << accountNumber << endl;
    cout << "Account Holder: " << holderName << endl;
    cout << "Current Balance: " << balance << endl;
}

int main()
{
    BankAccount acc1;
    acc1.setAccount(101, "Adam", 5000);
    acc1.deposit(1500);
    acc1.checkBalance();

    return 0;
}
```

BankAccount Example : Member Functions Defined Inside the Class

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
 private:
   int accountNumber;
   string holderName;
   float balance;

 public:
   void setAccount(int accNo, string name, float bal)
   {
       accountNumber = accNo;
       holderName = name;
       balance = bal;
   }

   void deposit(float amount)
   {
       balance = balance + amount;
       cout << "Amount deposited successfully." << endl;
   }

   void withdraw(float amount)
   {
      balance = balance - amount;
      cout << "Amount withdrawn successfully." << endl;
   }

   void checkBalance()
   {
      cout << "Account Number: " << accountNumber << endl;
      cout << "Account Holder: " << holderName << endl;
      cout << "Current Balance: " << balance << endl;
   }
};

int main()
{
   BankAccount acc1;     // Object creation

   acc1.setAccount(101, "Adam", 5000);   // Assign values

   acc1.deposit(1500);
   acc1.checkBalance();

   return 0;
}
```

What will be the output?

```
#include <iostream>
using namespace std;

int main()
{
    int i;
    for (i = 1; i <= 5; i++);
    {
        cout << i;
    }
    cout << i;
    return 0;
}
```

What will be the output?

```
#include <iostream>
using namespace std;

int main()
{
    int a[5] = {1, 2, 3, 4, 5};
    cout << a[1] + a[2] * a[3];
    return 0;
}
```

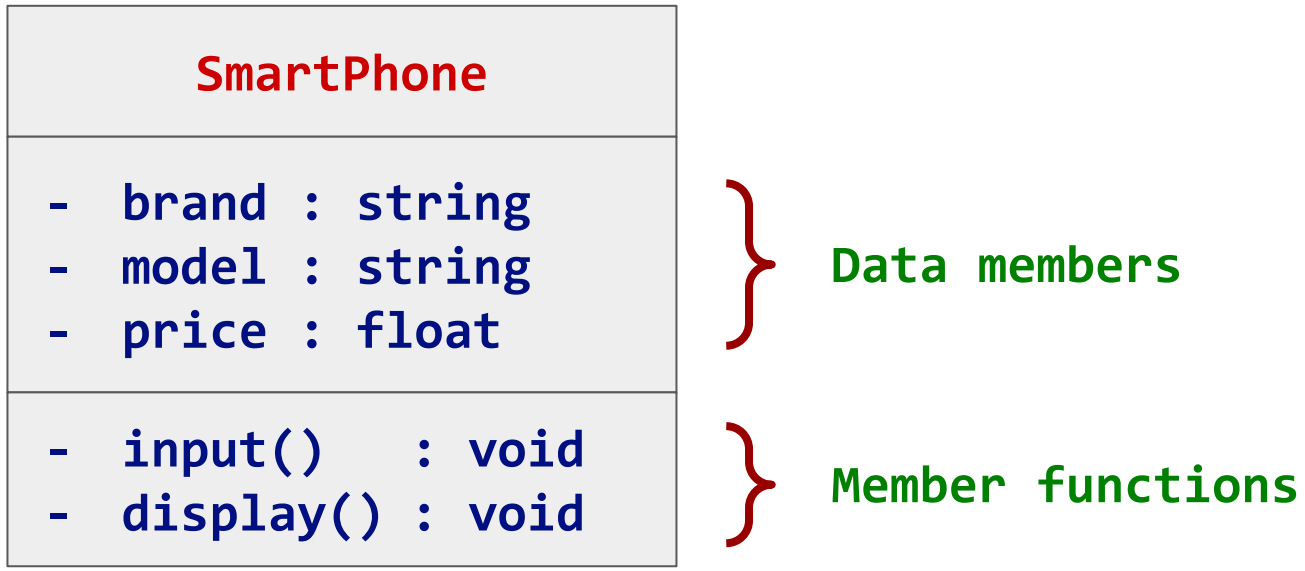
📝 Exercise 1

Create a class SmartPhone with the following details:

Access Specifier: Data Members -> private

Member functions -> public

Note: Member functions must be defined outside the class



Output Format:

```
Enter Brand: Samsung
Enter Model: Galaxy S23
Enter Price: 39999

SmartPhone Details
------------------
Brand: Samsung
Model: Galaxy S23
Price: 39999
```

Create an object of the class and access its data members and functions.

```cpp
#include <iostream>
#include <string>
using namespace std;

class SmartPhone
{
  private:
    string brand;
    string model;
    float price;

  public:
    void input();
    void display();
};

void SmartPhone::input()
{
    cout << "Enter Brand: ";
    cin >> ws;
    getline(cin, brand);

    cout << "Enter Model: ";
    cin >> ws;
    getline(cin, model);

    cout << "Enter Price: ";
    cin >> price;
}
```

```cpp
void SmartPhone::display()
{
    cout << "\nSmartPhone Details\n";
    cout << "Brand: " << brand << endl;
    cout << "Model: " << model << endl;
    cout << "Price: " << price << endl;
}

int main()
{
    SmartPhone s1;

    s1.input();
    s1.display();

    return 0;
}
```

📝 Exercise 2

Create a class Employee with the following details:

Access Specifier: Data Members -> private
Member functions -> public

Note: Member functions must be defined outside the class

Employee
- empId : int - name : string - year : float
- setData(int id, string n, float s) - displayData(): void

Output Format:

```
Employee Details
----------------
Employee ID: 101
Name: Albert
Salary: 35000
```

Note: Pass values to the object using parameters

```cpp
#include <iostream>
using namespace std;

class Employee
{
 private:
   int empId;
   string name;
   float salary;

 public:
   void setData(int id, string n, float s)
   {
       empId = id;
       name  = n;
       salary = s;
   }

   void display()
   {
       cout << "Employee Details";
       cout << "\n----------------";
       cout << "\nEmployee ID: " << empId;
       cout << "\nName: " << name;
       cout << "\nSalary: " << salary << endl;
   }
};
```

```cpp
int main()
{
   Employee e;

   e.setData(102, "John", 42000);  // passing values as parameters
   e.display();

   return 0;
}
```

📝 Exercise 3 :

Create a class Date with the following details:

Access Specifier: Data Members -> private
Member functions -> public

Note: Member functions must be defined outside the class

Date
- day : string - month : string - year : float
- getDate() : void - putDate() : void

} Data members

} Member functions

Output Format:

```
Enter day: 14
Enter month: 11
Enter year: 2025

Date: 14 / 11 / 2025
```

```cpp
#include <iostream>
using namespace std;

class Date
{
  public:
    int day;
    int month;
    int year;

    void getDate();
    void putDate();
};

void Date::getDate()
{
   cout << "Enter day: ";
   cin >> day;

   cout << "Enter month: ";
   cin >> month;

   cout << "Enter year: ";
   cin >> year;
}

void Date::putDate()
{
   cout << "\nDate: " << day << " / " << month << " / " << year << endl;
}
```

```cpp
int main()
{
   Date dt;

   dt.getDate();
   dt.putDate();

   return 0;
}
```

📝 Exercise 4

Create a class Employee with the following details:

Access Specifier: Data Members -> private

Member functions -> public

Employee
- empId : int - name : string - year : float
- inputData() : void - displayData(): void

Output Format:

```
Enter Employee ID: 101
Enter Name: Albert
Enter Salary: 35000

Employee Details
----------------
Employee ID: 101
Name: Albert
Salary: 35000
```

Note: Values are entered using member functions (no parameters).

```cpp
#include <iostream>
using namespace std;

class Employee
{
 private:
   int empId;
   string name;
   float salary;

 public:
   void input()
   {
       cout << "Enter Employee ID: ";
       cin >> empId;
       cout << "Enter Name: ";
       cin.ignore();
       getline(cin,name);
       cout << "Enter Salary: ";
       cin >> salary;
   }
   void display()
   {
       cout << "\nEmployee Details";
       cout << "\n----------------";
       cout << "\nEmployee ID: " << empId;
       cout << "\nName: " << name;
       cout << "\nSalary: " << salary << endl;
   }
};

int main()
{
   Employee e;

   e.input();
   e.display();

   return 0;
}
```

Difference between Class and Object

Class	Object
Class is the blueprint of an object. It is used to declare and create objects.	Object is an instance of class.
No memory is allocated when a class is declared.	Memory is allocated as soon as an object is created.
A class is a group of similar objects.	Object is a real-world entity such as book, car, etc.
Class is a logical entity.	Object is a physical entity.
Class can only be declared once.	Object can be created many times as per requirement.
Example of class can be car.	Objects of the class car can be BMW, Mercedes, jaguar, etc.

Constructor in C++

- A **constructor** in C++ is a **special member function** that has the **same name as the class.**
- Constructors are used to **initialize the objects** of a class.
- The constructor name must be the **same as the class name.**

  Reason: This helps the compiler identify it as a constructor.
- A constructor **does not have a return type,** not even **void**.

  Reason: It is called automatically and is mainly used for initialization.
- Constructors are **automatically invoked** when an object of the class is created.
- A constructor can be **defined inside or outside** the class definition.
- A constructor is usually declared in the **public section** of the class.

Syntax

Constructor (Inside the Class)

```
className (list_of_parameters)
{
   // constructor definition
}
```

Constructor (Outside the Class)

```
className :: className (list_of_parameters)
{
   // constructor definition
}
```

Example

```
class Student
{
   private:
      int rollNo;

   public:
      Student(int r)
      {
         rollNo = r;
      }
};
```

// constructor

```
class Student
{
   private:
      int rollNo;

   public:
      Student(int r); // constructor declaration
};

Student :: Student(int r)
{
   rollNo = r;
}
```

Types of Constructors

- In C++, constructors are mainly classified into the following types:

1) Default Constructor
    - A default constructor is a constructor **without parameters**.
    - It is called **automatically** when an object is created.
    - Used to initialize data members with **default values.**

2) Parameterized Constructor
    - A constructor with **parameters**
    - Used to initialize objects with **custom values**
    - Values are passed at the time of **object creation.**

Example 1: Default Constructor

When a book object is created **without details,** default values are assigned.

```cpp
#include <iostream>
using namespace std;

class Book
{
  private:
   int bookId;
   string title;
   float price;

  public:
   Book()    // Default Constructor
   {
       bookId = 0;
       title = "Not Assigned";
       price = 0.0;
   }

   void display()
   {
       cout << "\nBook ID: " << bookId;
       cout << "\nTitle: " << title;
       cout << "\nPrice: " << price << endl;
   }
};
```

```cpp
int main()
{
    Book b1;       // Default constructor called
    b1.display();

    return 0;
}
```

Output:

```
Book ID: 0
Title: Not Assigned
Price: 0
```

Example 2: Parameterized Constructor

When book details are known, values are passed **at the time of object creation.**

```cpp
#include <iostream>
using namespace std;

class Book
{
  private:
    int bookId;
    string title;
    float price;

  public:
    Book(int id, string t, float p)
    {
       bookId = id;
       title = t;
       price = p;
    }

    void display()
    {
       cout << "\nBook ID: " << bookId;
       cout << "\nTitle: " << title;
       cout << "\nPrice: " << price << endl;
    }
};
```

```cpp
int main()
{
   Book b2(101, "C++ Programming", 450.50);
   b2.display();

   return 0;
}
```

Output:

```
Book ID: 101
Title: C++ Programming
Price: 450.5
```

What will be the output?

```
#include <iostream>
using namespace std;

void change(int x)
{
    x = x + 10;
}

int main()
{
    int a = 5;
    change(a);
    cout << a;
    return 0;
}
```

What will be the output?

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string s = "C++ Programming";
    cout << s[4] << s[5] << s[6];
    return 0;
}
```

What will be the output?

```cpp
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    cout << (x > 5) && (x < 15);
    return 0;
}
```

What will be the output?

```
#include <iostream>
using namespace std;

int fun(int x)
{
    if (x > 0)
    {
        return x;
    }
    return x * -1;
}

int main()
{
    cout << fun(-5);
    return 0;
}
```

Templates in C++

Add two numbers (int, float)

```cpp
#include<iostream>
using namespace std;

int add(int a, int b)
{
    return a + b;
}

float add(float a, float b)
{
    return a + b;
}
```

```cpp
int main()
{
    int x = 10;
    int y = 20;

    float a = 20.20;
    float b = 10.20;

    cout << add(x, y) << endl;
    cout << add(a, b) << endl;

    return 0;
}
```

//without using templates

```
#include<iostream>
using namespace std;

int add(int a, int b)
{
   return a + b;
}

float add(float a, float b)
{
   return a + b;
}

int main()
{
   int x = 10;
   int y = 20;
   float a = 20.20;
   float b = 10.20;

   cout << add(x, y) << endl;
   cout << add(a, b) << endl;

   return 0;
}
```

//with using function templates

```
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b)
{
   return a + b;
}

int main()
{
   int x = 10;
   int y = 20;

   float a = 20.20;
   float b = 10.20;

   cout << add<int>(x, y) << endl;
   cout << add<float>(a, b) << endl;

   return 0;
}
```

Generic Programming in C++ (Templates)

- **Generic Programming** is a technique that **allows data types** (int, float, string, etc.) and **user-defined types** to be used as parameters in functions and classes.
- It helps in writing **reusable and efficient code**.
- The same algorithm can work with **multiple data types** without rewriting code.
- This avoids creating separate functions or classes for each data type (int, char, string, etc.).

Templates in C++

- **Templates** are a powerful feature of C++ used to implement **Generic Programming.**
- They allow us to write **one function or class** that works with **different data types.**
- The data type is decided **at compile time.**

Types of Templates

- C++ supports **two types** of templates:

    1. **Function Templates**
    2. **Class Templates**

How Templates Work?

- First, we declare a template.
- Immediately after the template declaration, we define a function or class.
- The function or class uses template parameters as data types.
- When the program runs, the compiler creates the required version based on the data type used.

1. Function Template

Defining a Function Template

- A **function template** starts with the keyword **template**.
- Template parameters are written inside **angle brackets** **< >**.
- After the template declaration, the **function definition** is written normally.

Syntax:

```
template <typename T>
T functionName(T parameter1, T parameter2, ...)
{
        // function body
}
```

Example:

```
template <typename T>
T add(T a, T b)
{
   return a + b;
}
```

- **T** is a **template parameter.**
- It represents a **data type** such as `int, float, char,` **etc.**
- `typename` is a keyword used to declare a **generic data type.**
- When a value is passed to **functionName(),**
  the compiler automatically creates a separate version of the function for that data type

Contd...

Calling a Function Template

Syntax:

```
functionName<dataType>(parameter1, parameter2, ...);
```

Example:

```
add<int>(10, 20);
add<float>(10.5, 20.5);
```

- `<float>` explicitly tells the compiler that **T** should be treated as `float`.
- The compiler generates a version of the function using the specified data type.
  - `add<int>` → creates a version of **add()** that works with `int`
  - `add<float>` → creates a version of **add()** that works with `float`

Function call based on data types

```
#include<iostream>

template<typename T>
T add(T num1, T num2) {
  return (num1 + num2);
}

int main() {
  ... ... ...

  result1 = add<int>(2,3);
  ... ... ...

  result2 = add<double>(2.2,3.3);
  ... ... ...

}
```

```
int add(int num1, int num2) {
    return (num1 + num2);
}
```

```
double add(double num1, double num2) {
    return (num1 + num2);
}
```

Adding Two Numbers Using Function Templates

```cpp
#include <iostream>
using namespace std;

template <typename T>
T add(T num1, T num2)
{
    return (num1 + num2);
}

int main()
{
    int result1;
    double result2;
    // calling with int parameters
    result1 = add<int>(2, 3);
    cout << "2 + 3 = " << result1 << endl;

    // calling with double parameters
    result2 = add<double>(2.2, 3.3);
    cout << "2.2 + 3.3 = " << result2 << endl;

    return 0;
}
```

Output:

```
2 + 3 = 5
2.2 + 3.3 = 5.5
```

📝 Exercise 1: Simple Calculator (Template Based)

Create function templates for:

- add
- subtract
- multiply
- divide

Use them for both int and float.

```cpp
#include <iostream>
using namespace std;

template <typename T>
T add(T a, T b)
{
    return a + b;
}

template <typename T>
T sub(T a, T b)
{
    return a - b;
}

template <typename T>
T mul(T a, T b)
{
    return a * b;
}

template <typename T>
T divi(T a, T b)
{
    return a / b;
}

int main()
{
    int x, y;
    float a, b;

    cout << "Enter two integers: ";
    cin >> x >> y;

    cout << "Addition (int): " << add<int>(x, y) << endl;
    cout << "Subtraction (int): " << sub<int>(x, y) << endl;
    cout << "Multiplication (int): " << mul<int>(x, y) << endl;
    cout << "Division (int): " << divi<int>(x, y) << endl;

    cout << "\nEnter two floating-point numbers: ";
    cin >> a >> b;

    cout << "Addition (float): " << add<float>(a, b) << endl;
    cout << "Subtraction (float): " << sub<float>(a, b) << endl;
    cout << "Multiplication (float): " << mul<float>(a, b) << endl;
    cout << "Division (float): " << divi<float>(a, b) << endl;

    return 0;
}
```